Loan Approval Prediction

1. Introduction

Loan approval is an important decision for financial institutions. Banks need to analyze multiple factors such as income, credit score, employment type and loan amount before approving a loan.

In this project, a Machine Learning classification model is developed to predict whether a loan application will be **Approved** or **Rejected** based on applicant information. The project builds and compares four different classification models using a structured Scikit-Learn preprocessing pipeline and selects the best-performing model for final prediction.

2. Problem Statement

The objective of this project is to build a machine learning model that predicts loan approval status using applicant data. The model analyzes the following features:

- Income
- Credit Score
- Loan Amount
- Years of Experience
- Gender
- Education
- City
- Employment Type

Using these features, the model predicts whether the loan will be:

- Approved
- Rejected

3. Dataset Overview

The dataset **loan_risk_prediction_dataset.csv** contains 5000 rows and 10 columns describing loan applicants.

JP

Feature Name	Description
Age	Age of the applicant
Income	Monthly income of the applicant
LoanAmount	Loan amount requested
CreditScore	Credit score of the applicant
YearsExperience	Work experience in years
Gender	Gender of the applicant
Education	Education level
City	Applicant's city
EmploymentType	Type of employment
Loan Status	Target variable (Approved / Rejected)

4. Import Libraries, Load Dataset & Data Cleaning

The dataset is loaded using pandas and initial inspection is performed to identify missing values, duplicate rows and the overall shape of the dataset.

a. Import Libraries & Load Data

```
import pandas as pd import
matplotlib.pyplot as plt import
seaborn as sns
plt.style.use('ggplot')

JP = pd.read_csv('loan_risk_prediction_dataset.csv')
print(JP.head(20))
```

b. Identify Missing Values & Duplicates

Column	Missing Values
Income	196
CreditScore	194
Education	198
All other columns	0

Duplicate rows found: **0** | Original dataset shape: **(5000, 10)**

c. Drop Irrelevant Column

The **YearsExperience** column was found to have no meaningful predictive value for the target variable and is removed from the dataset.

```
JP.drop(columns=['YearsExperience'], inplace=True)
```

d. Remove Invalid (Negative) Values

Some records contained negative values in the **LoanAmount** column, which is not logically valid.
These records are filtered out.

```
JP = JP.query(" LoanAmount > 0 ")
```

5. Feature Selection and Preprocessing Pipeline

a. Splitting Features (X) and Target (Y)

```
x = JP.drop(columns=('LoanStatus'),axis=1)
y = JP['Loan Status']
```

b. Building a Column-wise Preprocessing Pipeline

Instead of manually encoding columns, this project uses a Scikit-Learn **ColumnTransformer** combined with **Pipeline** objects to handle numeric and categorical columns separately and consistently:

- **Numeric columns** — missing values filled with the **median**, then scaled using **StandardScaler**.
- **Categorical columns** — missing values filled with the **most frequent** value, then encoded using **OneHotEncoder**.

```
numeric_col = x.select_dtypes(include=['int64','float64']).columns categorical_col =
x.select_dtypes(include=['object']).columns

numeric = Pipeline([('imputer', SimpleImputer(strategy='median')),     ('scaler',
StandardScaler())])

categorical = Pipeline([('imputer', SimpleImputer(strategy='most_frequent')),
('encoder', OneHotEncoder(handle_unknown='ignore'))])

preprocessing = ColumnTransformer([('Numeric', numeric, numeric_col),

('Categorical', categorical, categorical_col)])
```

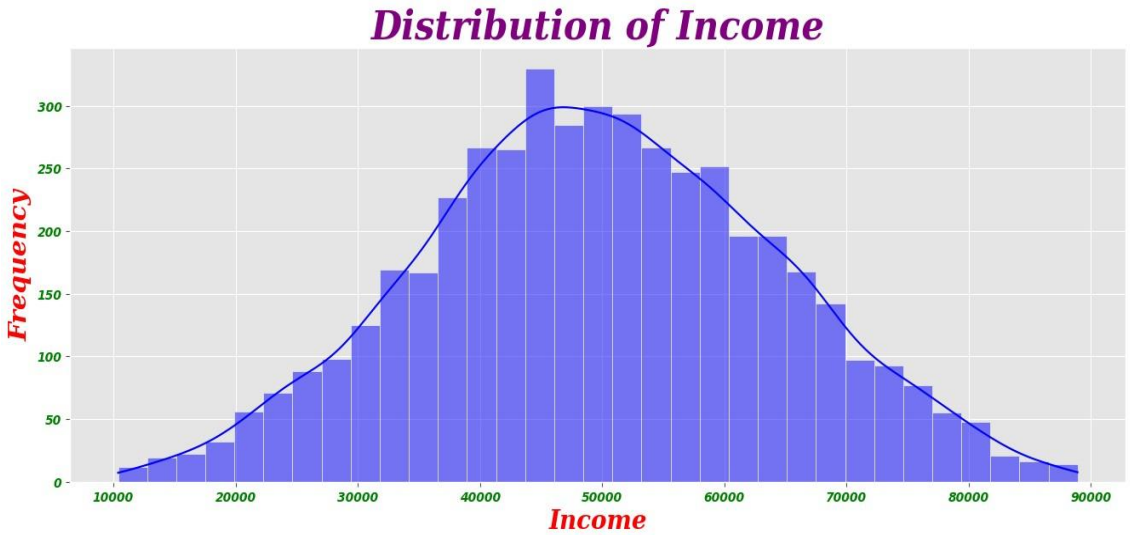
- **This pipeline-based approach ensures the same preprocessing logic is applied consistently to training and test data, and prevents data leakage.**

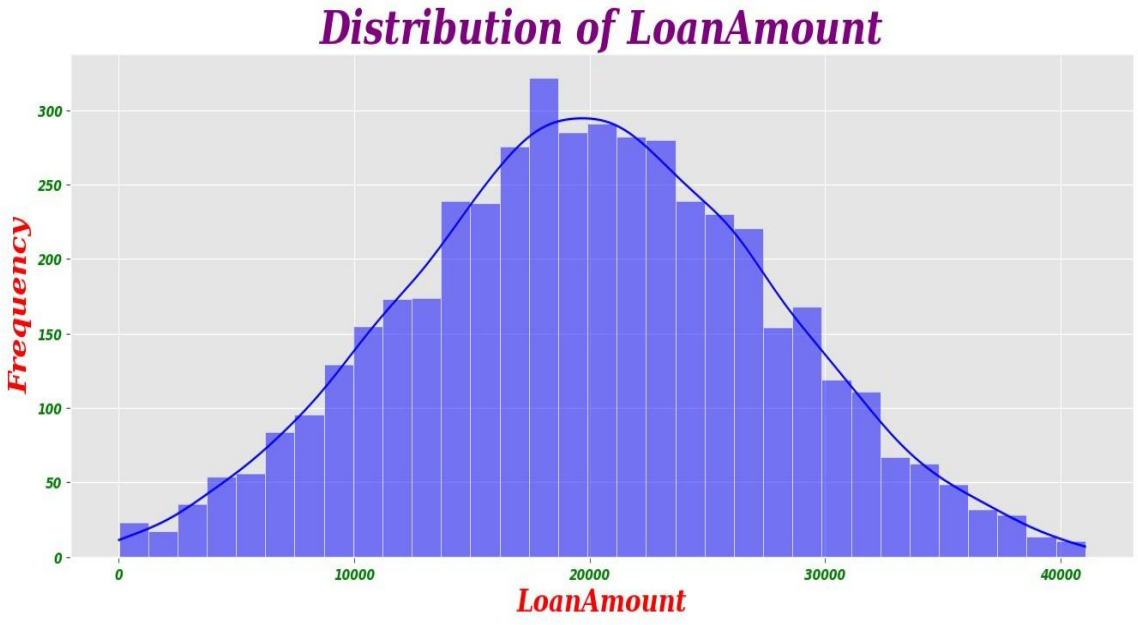
6. Exploratory Data Analysis (EDA)

Several visualizations are generated to understand feature distributions and their relationship with the target variable.

- **Histograms with KDE** — plotted for every numeric column to view its distribution shape.
- **Scatter plots** — Income vs CreditScore and LoanAmount vs CreditScore to explore linear relationships.
- **Count plots** — Education, Gender, City and EmploymentType are plotted against Loan Status to compare approval/rejection patterns across categories.
- **Pie chart** — visualizes the overall Approved vs Rejected distribution in the dataset.

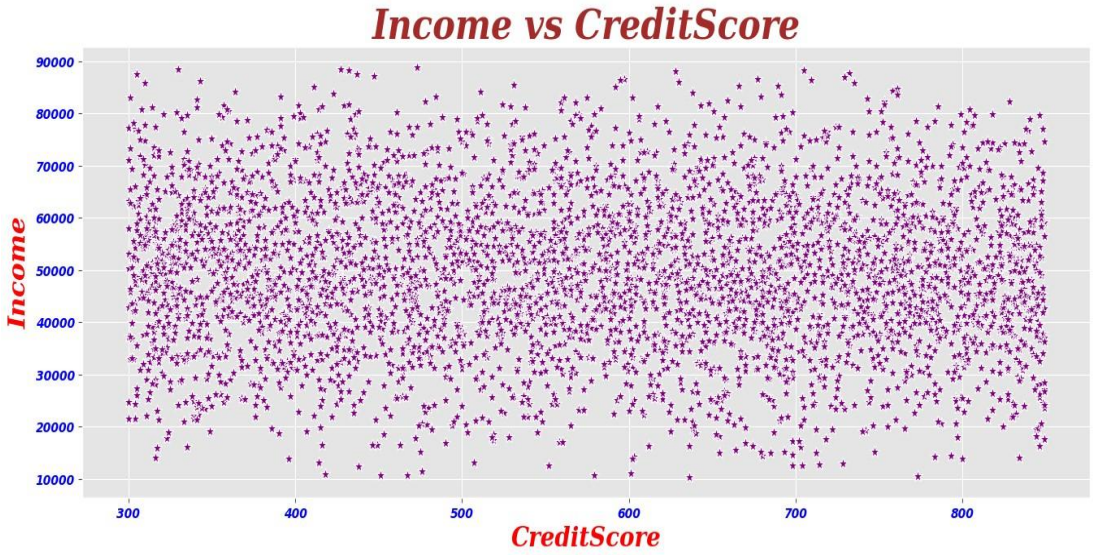
a. Frequency Distributions (Histograms)

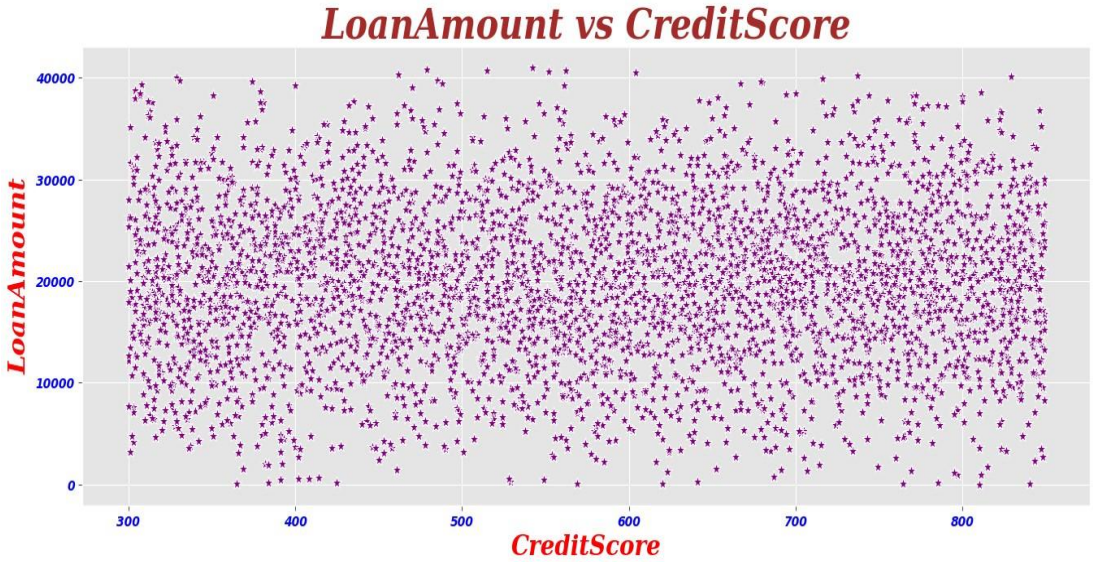




JP

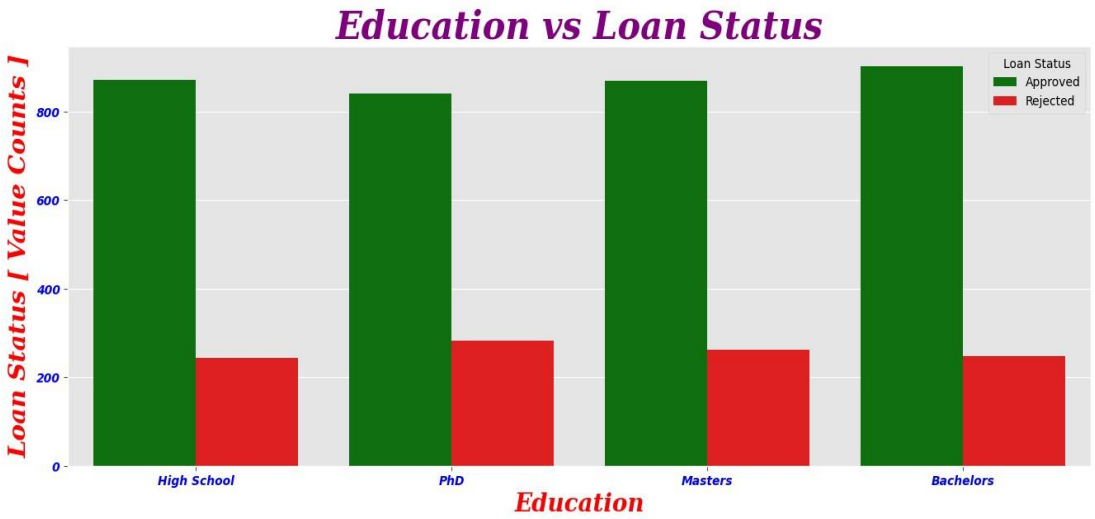
b. Relationship with CreditScore (Scatter Plots)

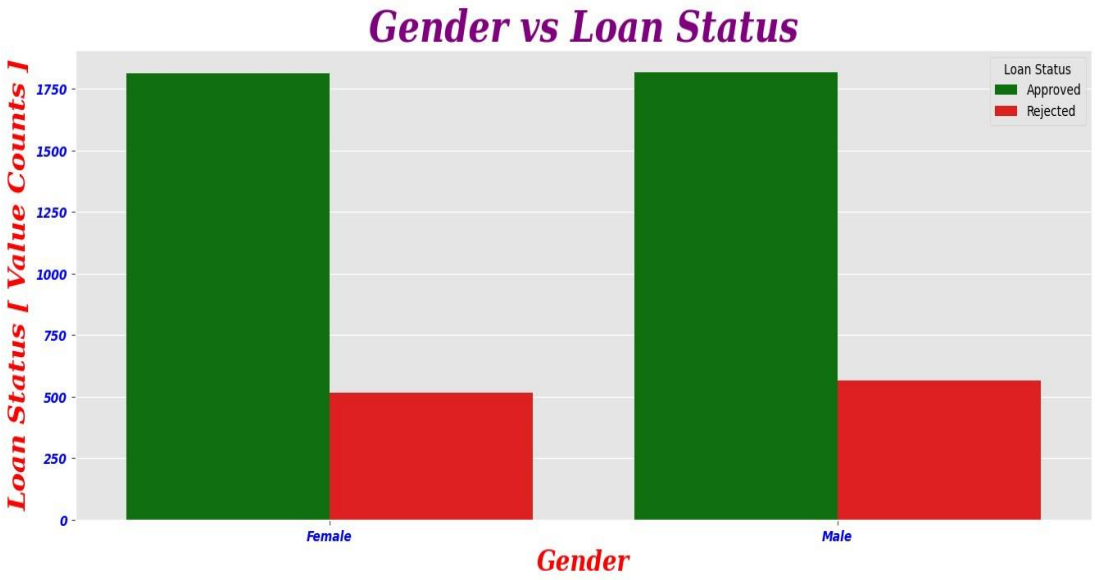


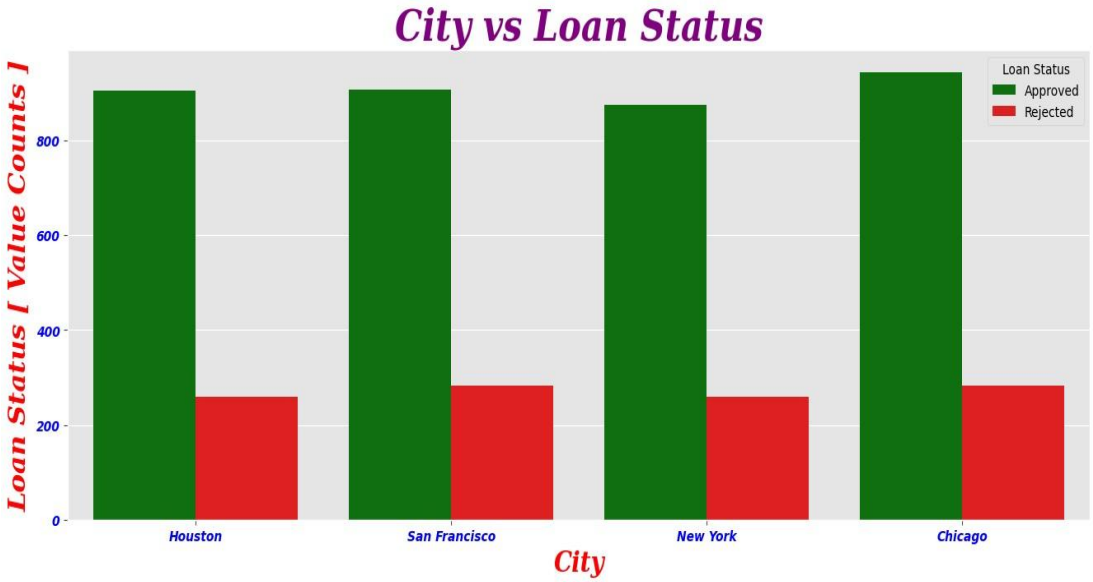


JP

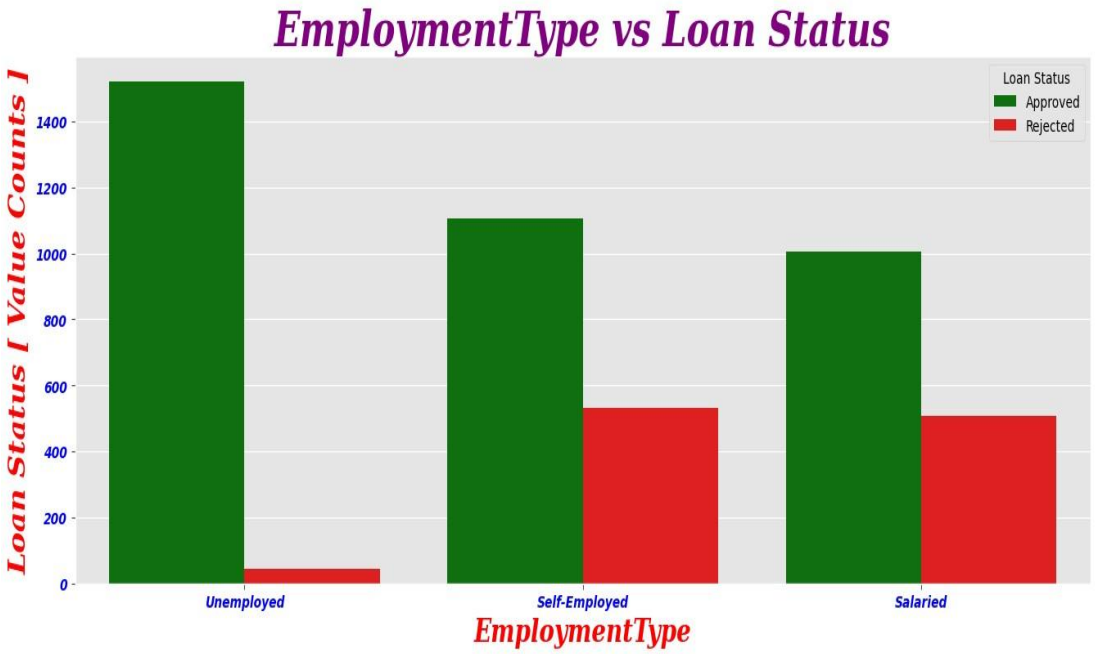
c. Categorical Features vs Loan Status (Count Plots)



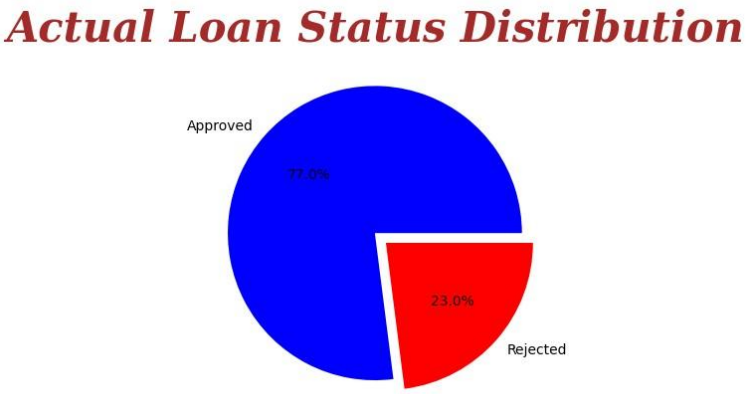




JP



d. Overall Loan Status Distribution



7. Train-Test Split & Model Building

The dataset is split into training (80%) and testing (20%) sets. Four different classification models are trained, each wrapped inside a Pipeline together with the shared preprocessing step:

- Decision Tree Classifier (max_depth=5, criterion='gini')
- Logistic Regression (max_iter=1000)
- Random Forest Classifier (n_estimators=100)
- Support Vector Machine — SVC (kernel='rbf', class_weight='balanced')

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=98)

dt=DecisionTreeClassifier(max_depth=5,criterion='gini',random_state=77) DT_model =
Pipeline([('Preprocessing', preprocessing), ('Model', dt)]) DT_model.fit(x_train, y_train)

lr = LogisticRegression(max_iter=1000)

lr_model=Pipeline([('Preprocessing',preprocessing),('Model',lr)])

rf = RandomForestClassifier(n_estimators=100, random_state=67)
rf_model=Pipeline([('Preprocessing',preprocessing),('Model',rf)])

svm =SVC(kernel='rbf',C=1.0,gamma='scale',class_weight='balanced',random_state=34)
svm_model = Pipeline([('Preprocessing', preprocessing), ('Model', svm)])
```

8. Model Accuracy Comparison

The accuracy of each trained model was calculated on the test set:

Model	Accuracy
Decision Tree	97.61 %
Logistic Regression	88.24 %
Random Forest	96.61 %
SVM	90.89 %

✓ Final Model Selected: Decision Tree Classifier

The Decision Tree was chosen as the final model because it matched the highest accuracy achieved while remaining simple and more interpretable.

9. Model Evaluation

The selected Decision Tree model was evaluated in depth using train/test scores, cross-validation, and standard classification metrics.

a. Train / Test / Cross-Validation Scores

Metric	Score
Train Score	0.9735
Test Score	0.9761
Average CV Score (5-fold)	0.9677

Cross-validation fold scores: [0.9722, 0.9695, 0.9629, 0.9695, 0.9642].

The closest of the train, test and CV scores indicates the model generalizes well and is **not overfitting**.

b. Classification Report

Class	Precision	Recall	F1-score	Support
Approved	0.98	0.99	0.98	779
Rejected	0.97	0.92	0.93	216
Accuracy			0.97	995
Macro Avg	0.97	0.94	0.95	995
Weighted Avg	0.97	0.97	0.97	995

c. Confusion Matrix

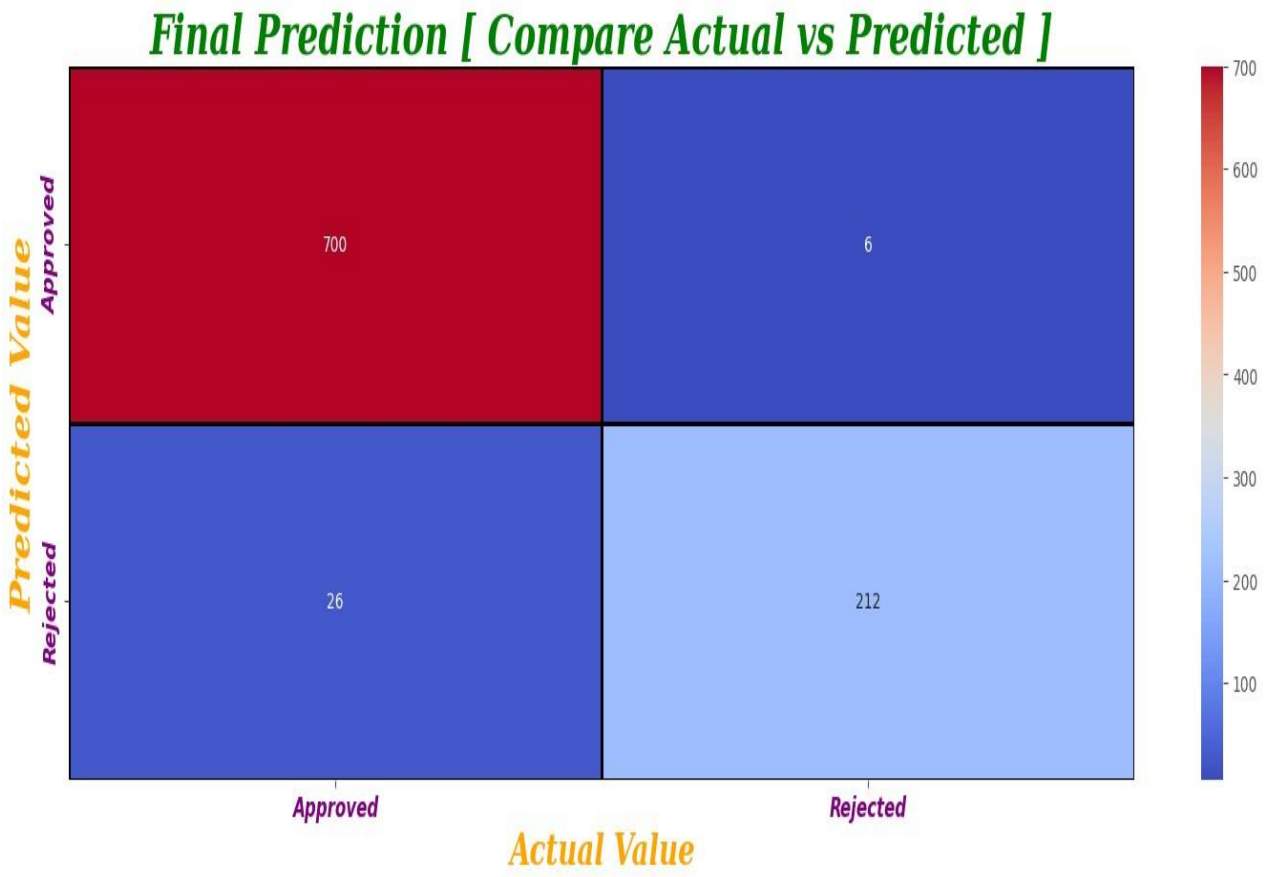
	Predicted Approved	Predicted Rejected
Actual Approved	769	10
Actual Rejected	18	198

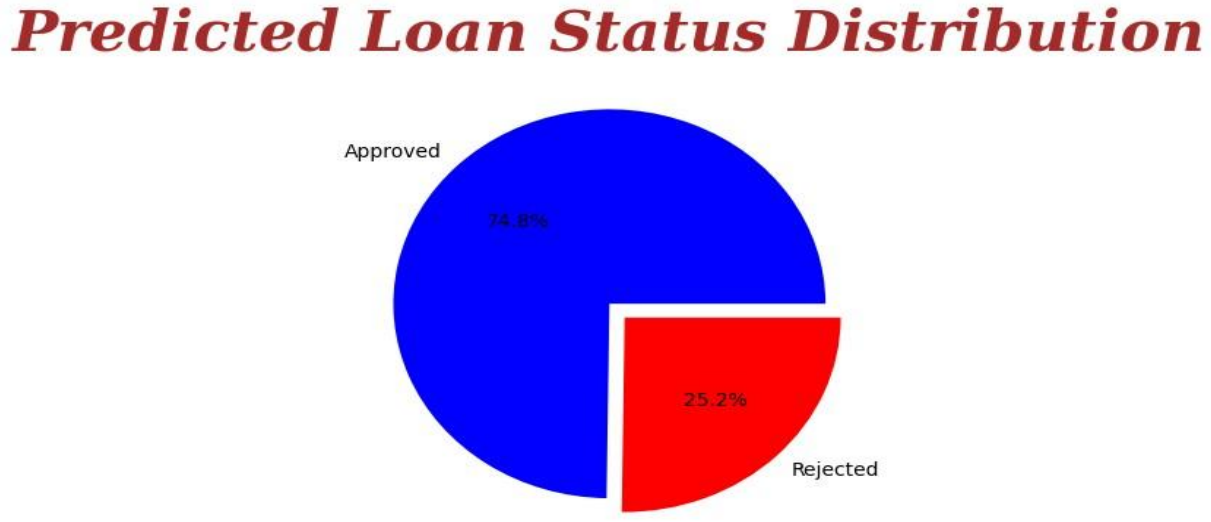
769 loans were correctly predicted as Approved, and 198 were correctly predicted as Rejected. Only 10 and 18 cases were misclassified respectively, confirming strong predictive performance.

10. Final Prediction

The final section visualizes the model's predicted loan status distribution and compares it against the actual test-set outcomes.

- A pie chart shows the predicted distribution of Approved vs Rejected loans in the test set.
- A heatmap of the confusion matrix visually compares actual vs predicted values, using 'Approved' and 'Rejected' .





11. Conclusion

- This project demonstrates an end-to-end machine learning workflow for predicting loan approval decisions from raw data to a deployable model.
- A structured Scikit-Learn Pipeline with a ColumnTransformer was used to handle missing values, scaling and encoding consistently across numeric and categorical features.

JP

- Four classification models were trained and compared; the Decision Tree Classifier was selected as the final model, achieving ~97.6% test accuracy with no signs of overfitting.
- The project highlights the importance of proper data cleaning, exploratory analysis, and model evaluation when building machine learning solutions.

Thank You

JP